

Lab Exercise 3: Secure Dynamic Banking System using OOP

Objective: To implement a modular, secure Banking System in Java leveraging encapsulation, constructor initialization, inheritance mechanisms, and runtime polymorphism.

Aim: To design an abstract base class BankAccount and concrete sub-classes that implement variable transaction behaviors dynamically.

Theory: Encapsulation hides inner operational data states by routing actions through public getters and setters. Runtime polymorphism allows a base class reference variable to seamlessly manage distinct sub-class methods (e.g., standard vs. savings calculations) using late binding.

Algorithm / Procedure:

1. Design an abstract base class BankAccount containing encapsulated fields (accountNumber, balance).
2. Create concrete classes SavingsAccount (with interest accrual) and CurrentAccount (with overdraft limits).
3. Override a generic withdraw(double amount) block across child instances to handle specific transaction boundaries.
4. Process dynamic inputs to trigger late-binding validation transactions.

Program Code: Problem3_BankingSystem.java

```
import java.util.*;
```

```
abstract class BankAccount {
```

```
    protected String accNo;
```

```
    protected double balance;
```

```
    public BankAccount(String accNo, double balance) { this.accNo = accNo; this.balance = balance; }
```

```
    public abstract void withdraw(double amt);
```

```
    public double getBalance() { return balance; }
```

```
}
```

```
// TODO: Implement SavingsAccount and CurrentAccount child extensions here
```

```

public class Problem3_BankingSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Select Account Type (1-Savings, 2-Current): ");

        int choice = sc.nextInt();

        System.out.print("Enter Acc No and Initial Balance: ");

        String num = sc.next(); double bal = sc.nextDouble();


        BankAccount account = (choice == 1) ? new SavingsAccount(num, bal) : new
        CurrentAccount(num, bal);

        System.out.print("Enter withdrawal amount: ");

        double amt = sc.nextDouble();

        account.withdraw(amt);

        System.out.println("Remaining Balance: " + account.getBalance());

        sc.close();

    }

}

```

(Note: Students must write the missing classes so that the polymorphic driver code executes correctly.)

Sample Input / Output:

Input:

1

SAV101 500.0

600.0

Output:

Error: Insufficient Funds. Savings cannot go below 0.

Remaining Balance: 500.0

Evaluation Criteria (Total Marks: 10):

- Program compiles and runs successfully (2 Marks)
- Valid use of abstraction and encapsulation principles (3 Marks)
- Proper execution of runtime polymorphism patterns (2 Marks)
- Conditional edge-case logic parsing constraints (2 Marks)
- General code documentation clarity standards (1 Mark)